

EGOI 2025 Editorial - Giftboxes

Task Author: Anja Dožić

July 16, 2025

The problem

The organizers are giving gift boxes to competitors and want to gift one box to each team. There are T teams and N competitors waiting in line for gifts. The problem is that girls in a line are mixed up such that the girls from the same team might not be standing next to each other, and there may be multiple girls per team in the line. The organizers have to pause the gifting process once and let some girls pass without receiving the gift boxes. The goal is to maximize the number of teams receiving gift boxes, but to avoid giving more than one gift box to the same team (we can give a gift box to at most one competitor from one team, while others must be passed).

Subtask 1: $N = T + 1$

In this subtask, only one team has two members in the line, while the remaining all have exactly one.

Here, it is possible to gift every team by passing one of two participants from a team that appears twice. This is true because when we remove that one participant from an array, we will have an array with all distinct elements (because all other participants are from different teams).

To find the team that appears twice, go through the given array and mark every element once it appears. This can be done by maintaining a separate array b of boolean values, where the value of $b[i]$ is changed to *true* once the number i appears. If $b[i]$ is already true, that means that i has already appeared in the array once and we found the second appearance, so we can print index i twice to remove that participant.

Subtask 2: $N = 2 \cdot T$ and every team will appear once in each half

In this subtask, it is also possible to gift every team. This can be done by choosing a half and passing it completely. This strategy is correct because it is guaranteed that there is exactly one appearance of each team in each half, so if we remove one half, the other will contain all elements from 0 to $T - 1$ exactly once. Therefore, printing 0 and $N/2$ or $N/2 + 1$ and N solves this subtask.

Subtask 3: $T, N \leq 500$

This subtask can be solved by iterating through all possible pause intervals and choosing the best one. In detail, this works as follows:

Iterate through all possible start and end points of the pause interval. For a specific interval, we have to check whether the array consists of distinct numbers after removing that interval. This can be done using a similar strategy as in the first subtask. In the end, we output the shortest interval that fulfilled this condition.

Note that the time complexity of this solution is $O(N^3)$, which is fast enough this subtask, but is too slow for the remaining subtasks.

Subtask 4: $N = 2 \cdot T$ and every team will appear twice

In this subtask, the problem boils down to finding the shortest subarray that contains all elements at least once. This is sufficient because if all elements appear twice, it is necessary to remove at least one of the appearances for each of them.

This subarray can be found using the two-pointer method. At the beginning, the left pointer is on the first element, while the right pointer is on the first appearance of the element that appears last. The subarray between those two pointers contains all elements at least once (since it contains the first appearance of each element).

Now, we need to move the pointers in a way such that the array between them always satisfies this condition. This can be done in the following way:

In each round, first move the left pointer right by one. Then, we removed the first appearance of an element from the subarray. Because we need the subarray to contain all elements, we need to make sure that it will contain the second appearance of the removed element. This is done by moving the right pointer right until the subarray contains the second appearance of the removed element.

In the end of each round, we calculate the length of the constructed subarray. This process ends once the left pointer reaches the second appearance of some element. Finally, the shortest valid subarray is chosen.

Subtask 5: $M, N \leq 5000$

In this subtask, we are aiming to reduce our time complexity of Subtask 3 from $O(N^3)$ to $O(N^2)$. To do that, we want to accelerate the check whether a subarray satisfies the conditions.

First, count for each element how often it appears. Then, we again iterate through all subarrays. First, fix the start of the subarray and iterate through all possible endings. While doing this, we can maintain an array b , such that $b[i]$ is the number of appearances of the i -th element in the subarray. Also, maintain a counter c of the number of elements that appear more than once outside of the subarray. For an element i , this is the case precisely if the difference between $b[i]$ and the total number of appearances is 1 or more. A subarray satisfies the condition if and only if $c = 0$. Both b and c can be updated in constant time when the ending of the subarray is moved right by one. In the end, the shortest subarray that satisfies the condition is chosen.

Subtask 6: No additional constraints

For the full solution, the following observation is needed: if an element appears more than twice, every appearance between the first and the last will be in the subarray we intend to remove.

Firstly, why is this true? Let us assume that we have one element which appears more than twice, and there is an appearance that is not first nor last and that is not in the chosen subarray. The subarray must be either left or right from the appearance of the element. If it is left, then we know that the last appearance of the element, as well as the one we are looking at, will be in the array that is left when we remove the chosen subarray. That means that the conditions of the task are not satisfied. If it is right, we get an analogous contradiction.

We will call those appearances (those which are neither first nor last) *middle appearances*. Using the observation, we can compress the given array in the following way: we will remove the subarray bounded by the first and last middle appearances among all elements because we know those numbers will definitely be in the chosen subarray.

Now, we simplified the full task to a subtask 4 with two additional constraints:

1. Elements appear at most twice, so there might be some that appear once.
2. We must include the removed elements in the chosen subarray.

Since the elements that appear at most once can be ignored when choosing the subarray, we can use a similar algorithm as in subtask 4, and then unite the found interval with the interval that we already removed.

EGOI 2025 Editorial - Dark Ride

Task Author: Ivan Gaspardy

July 16, 2025

Introduction

In this interactive problem, there are N rooms and N light switches that each turn on the light in a distinct room (but we do not know the matching). The goal is to find out which switches belong to the first and last room. In each query, you can turn on any number of switches. The answer to a query is the number of changes between a dark room and a lit room.

Subtask 1 ($N = 3$)

In this subtask, there is a starting room, an ending room, and a middle room between them. Suppose the light is only on in the middle room. The passengers will scream once when entering the room, and once when exiting it. What happens if the light is only on in the last room? What happens when the light is only on in the first room? In these cases, the passengers will only scream once, because the rollercoaster won't enter the first room, and it won't exit the last room. Thus we can check in a single query, whether or not a particular switch belongs to the middle room by setting only that switch on. If that switch belongs to the middle room, the query will return 1. Otherwise it will return 2. Thus we make three queries (100, 010, and 001) to find the rooms at both ends.

Subtask 2 ($N \leq 30$)

We can extend the idea from the previous subtask to solve this. In one query, we can check if a particular switch controls one of the end rooms, by making 30 queries, each of which only has one switch on. The two queries that return 1 must have had one of the end switches on.

Subtask 3 ($p_0 = 0$)

Let's call room 0 the first room. In this subtask, we never turn on switch 0, and thus, the first room stays dark. We further extend our observation from subtask 1. How many times will the passengers scream? First, suppose the last room is dark as well. We know that they'll scream the first time they enter a lit room. We also know that every time they enter a lit room, they'll eventually enter a dark room, since the last room is dark. Hence they must scream an even number of times. If the last room is lit, then they will scream an odd number of times since in the end, they do not go from a lit room to a dark room again. We can use this knowledge to binary search which switch controls the light in the last room. This will take $\log_2 N$ queries, which is no more than 15. In general, if we know where one output switch is, then we can binary search for the other one in at most 15 queries.

Subtask 4 (N is even, and one output switch in each half)

Without loss of generality, let's say the output switch in the first half of the switches controls the first room, and the output switch in the second half controls the last room. We can extend our binary search approach from subtask 2. Previously we knew exactly where the switch for the first room was, but now we only have a range for where it could be. This actually doesn't stop us from applying the same strategy as Subtask 2 to locate the switch that controls the last room, since we can set all of the switches in the first half to dark, and be assured that the first room is dark, so we're free to perform our binary search on only the switches in the second half. Once we've located one of the output switches, finding the second one is a matter of repeating the algorithm, but in reverse. That is, set the entire second half to dark, and run the binary search in the first half. Each binary search will take no more than 14 queries, so we will be able to locate both switches in 28 queries.

Subtask 5 ($N \leq 1,000$)

Here we are not guaranteed anything about the locations of the significant switches. What would be ideal is if, like Subtask 4, we somehow managed to split all of the switches into two groups such that each group has only one significant switch in it. We discussed earlier that the query results will be odd if and only if the end rooms ended up with different light settings, so we would like to create a circumstance where an odd query is guaranteed since once we have that, we can apply our solution from Subtask 4. What happens if we split the array down the middle into a first half and a second half, and we make a query where we only set the first half on? If the result is odd, great! Otherwise, we know that either both switches are in the first half, or both are in the second half. We can actually repeat this strategy on each half of the array. We split each half into halves, making quarters, and we set the first and third quarters to be on (that is the first half of the first half, and the first half of the second half). Below is a diagram where $N = 8$.

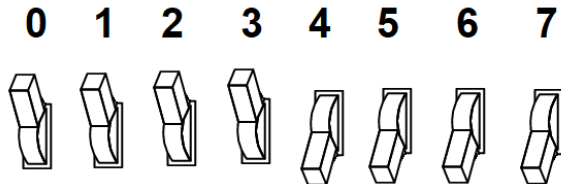


fig. a

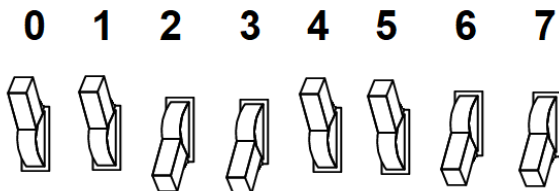


fig. b

fig. a is our first query, and fig. b is our second query. Now if we end up with an odd result, great! Otherwise, we know that the two significant switches must be in the same quarter of the array, so we can cut each of the quarters in half, making eighths, and repeat! This strategy will guarantee that we end up with an odd result in no more than $\log_2 N$ queries, which for this subtask, is not more than 10. Once we've got

an odd query, we've successfully split the array into two groups such that each group contains one significant switch, so we can apply our solution to Subtask 4. Note that because $N \leq 1,000$, our Subtask 4 algorithm will take only 20 queries, so we fit in under 30 total queries.

Full Solution

For the full solution, we find that we only actually need to run half of the Subtask 4 algorithm. That is, we only need to use binary search to locate one of the significant switches. In Subtask 5, we stopped subdividing the switches into smaller and smaller groups as soon as we hit an odd query. If we don't do this, and we instead let it run through 15 iterations, and then use binary search to find one of the significant switches, we will actually have all the necessary information to locate the second significant switch. We can do this by going over our initial 15 queries. Since we now know where one of the significant switches is (let's call this switch A), we can tell based on the query result, whether or not the two significant switches had the same light setting or not. It happens that there will always be exactly one switch that is consistently set the same as switch A when the result is even, and set different from switch A when the result is odd. Thus it must be switch B , and we've located both the end switches in no more than 30 queries.

EGOI 2025 Editorial - Monster Go

Task authors: Dario Martinez Ramirez, Manuel Torres Cid

July 16, 2025

In this problem, you have to output N lists each consisting of 12 distinct numbers from 0 to 49. A list wins if all its numbers have been enumerated first. You have to make sure that no matter in which order the numbers are enumerated, there will always be a unique winner. A general observation here is that when you have two lists A and B , for each number k in $A \cap B$, you also need a list that is a subset of $A \cup B \setminus \{k\}$ - this makes sure that we can never run into a situation where A and B both only still need number k for winning.

What is special about this problem is that the test cases are public - there are many different approaches on how to solve some test cases, it is even possible to solve the smaller ones by hand to score some points. In the following, we explain some approaches on how to construct such lists and then show how to combine those approaches into a full solution. By only finding some but not all of these constructions, one can obtain a partial solution.

Disjoint Lists. The simplest way to construct such a set of lists is to just use disjoint lists. This approach can solve $N = 1, N = 2, N = 3, N = 4$.

Rotations. We can create 13 lists using 13 numbers: create list i by removing the i -th number from the set. There will be a unique winner now: assume that the number k is the last number that makes 2 lists A and B win at the same time: then there is also a list $(A \cup B) \setminus \{k\}$ which would then win before A and B , so we have a contradiction. This idea can solve $N = 13$, and by iteration we can also get $N = 26$ and $N = 39$

Generalized Rotations. We can generalize the idea from before: Take any divisor d of 12. Define

$$S_1 = \{1, \dots, d\}, S_2 = \{d+1, \dots, 2d\}, \dots, S_{n/d+1} = \{13, \dots, 12+d\}$$

Then, construct list

$$l_j = (\bigcup S_i) \setminus S_i$$

. The correctness proof is the same as for the simple rotation. When choosing $d = 1$, we obtain back the simple rotation. This idea can solve cases $N = 12/d + 1$ for any divisor d of 12 (and can again be iterated)

Generalizing further. Note that we can also add additional subsets to a generalized rotation: For some positive integer k and a divisor d , define

$$S_1 = \{1, \dots, d\}, \dots, S_{n/d+1} = \{13, \dots, 12+d\}, \\ \dots, S_{n/d+k} = \{13 + (k-1)d, \dots, 12 + k \cdot d\}$$

Then, we can create $\binom{n/d+k}{k}$ lists, each one containing all numbers in all of the subsets except k of them.

Combining approaches. More solutions can be obtained by combining the approaches from before: Define a *block* of size N with cost c as a solution for N that uses c distinct numbers.

If we now want to construct a solution for a new N , we have to find blocks $(N_1, c_1) \dots (N_k, c_k)$ such that $N_1 + \dots + N_k = N$ and $c_1 + \dots + c_k \leq 50$. There are two ways to find such a combination:

Note that since every block has a cost of at least 12, we can combine at most 4 blocks. Thus, we can iterate through all possible combinations of four blocks to find a solution. Instead, it is also possible to use a knapsack DP.

EGOI 2025 Editorial - Wind Turbines

Task authors: Chur Zhe Yaw, Shi Wei Tia

July 16, 2025

Introduction

In this problem, we are given an undirected weighted graph and have to answer queries that ask for the minimum cost of connecting each node to an arbitrary node in the range $[l_i, r_i]$.

Subtask 1 $M = N - 1$ and $v_i = u_i + 1$

In this subtask, the graph is a path. For a query $[l_i, r_i]$ the optimal solution is to add all edges $(0, 1), (1, 2) \dots (l_i - 1, l_i)$ and $(r_i, r_{i+1}), \dots (N - 2, N - 1)$. To compute this cost efficiently for each query, we can precompute it using prefix sums. The runtime is $\mathcal{O}(N + Q)$

Subtask 2 $N, M, Q \leq 2,000$ and $\sum(r_i - \ell_i + 1) \leq 2,000$

In this subtask, the constraints are small enough for running a minimum spanning tree algorithm like Kruskal or Prim for each query. To account for the fact that some nodes are connected to the shore for free, we have to add a path of edges with weight 0 connecting the nodes in $[l_i, r_i]$ before running the MST (Minimum Spanning Tree) algorithm. This solution runs in time $\mathcal{O}(Q \cdot M \log M + \sum(r_i - \ell_i + 1))$

Subtask 3 $r_i = \ell_i + 1$

In this subtask, there are exactly two nodes connected to the shore for each query. That means, compared to the MST, one edge can be removed for each query. The edge that should be removed is the heaviest edge on the path from ℓ_i to r_i . To efficiently find this edge for every query, we can use binary lifting:

First, root the tree arbitrarily. Then for every node v , and for every path length p that is a power of two, compute the heaviest edge that you encounter on the path from node v if you go p steps upwards. Also, save the node that you get to if you go p steps upwards from v . Both values can be computed in time $\mathcal{O}(N \log N)$ using dynamic programming.

Then, to answer a query (ℓ_i, r_i) , we can compute the heaviest edges on the path from ℓ_i to the lowest common ancestor (LCA) and from r_i to the LCA: we can "walk up" to the LCA in the same way as done in the standard binary lifting LCA algorithm and take the max of the heaviest edges found on the way. The runtime is $\mathcal{O}(N \log N + Q \log N)$

Subtask 4 $1 \leq c_i \leq 2$

This subtask can be solved using similar observations as the 100pt solution (see below), but with significant simplifications. In particular, from the size of the interval $[\ell, r]$ we know how many connections need to be paid for in total. Hence, we only need to find how many connections of cost 2 need to be used. This can also be calculated by figuring out how many connected components there are in the graph consisting of only the $c = 1$ edges for each query.

Subtask 5 $\sum(r_i - \ell_i + 1) \leq 400,000$

Suppose $q_i = [\ell_i, r_i]$ is one of the queries. The cost of this query is the cost of the MST of the original graph minus the cost of some edges. In particular, exactly the edges in the MST that joined two sets which both had a turbine from $[\ell_i, r_i]$ are the ones in the MST that we don't have to pay for. To solve the query, we just have to find the cost of those edges.

We use Kruskal's algorithm to construct MST of the input graph. We can slightly extend the union-find / disjoint-set-union data structure used by Kruskal's algorithm to compute the MST. Before we do so, we read in all the queries, and store at each node a list of queries it is part of. Since $\sum(r_i - \ell_i + 1) \leq 400,000$, we don't store too many values. When merging two sets, we can also merge the list of query indices to store at the union set. We do so by going over all the query indices stored at the smaller set and: (1) check if they are also stored in the larger set (if so, we can record this information and update the cost saving for that query) and (2) add them to the list in the larger set. In total, this takes $O(N \log N)$ time, since whenever a vertex is processed as part of a smaller set, it will after the union operation belong to a set at least twice the size.

Alternative Solution. One can also process the each query in time $O((r_i - \ell_i + 1) \log n)$ time, using *Virtual Trees* (see e.g., <https://codeforces.com/blog/entry/140066>).

Subtask 6 $\ell_i = 0$

There are multiple ways of getting this subtask:

- Doing a simplified version of the 100pt solution (see below), where one can skip the fenwick tree since all queries start at $\ell = 0$.
- Solving offline dynamic MST, where you add 0 edges from i to $i + 1$ one at a time. See e.g., <https://codeforces.com/blog/entry/105192> for how to solve the offline dynamic MST problem.
- Using Heavy Light Decomposition, one can compute the change between the query $[0, r]$ and query $[0, r + 1]$ by figuring out which edge should no longer be used in the MST, and removing it.

We leave the details up to the reader to figure out :)

Full Solution

For full points on this problem, we need a few ideas and data structures.

Preprocessing. Let us first run Kruskal's MST algorithm on the input graph, before processing any queries.

While doing so, we can construct the following *Kruskal tree* with $2n - 1$ nodes, consisting of:

- n leaf nodes, each representing a city,
- $n - 1$ internal nodes, each representing a power line used in the original MST.

To construct this tree:

1. Sort all power lines in non-decreasing cost.
2. Process each power line in order:
 - If u_i and v_i are already connected, skip this power line.

- Otherwise, create a new internal node y and make it the parent of the roots of u_i and v_i .
- Then, set the root of u_i and v_i to y .

Observe that any power line not in the original MST will never be built in any scenario; hence we can safely ignore those power lines.

Now, let y_i be the node in the tree corresponding to power line i . It will be built if and only if y_i has at least one direct child such that all leaves in that child's subtree do not have a power plant.

In other words, power line i will **not** be built if and only if both direct children of y_i have at least one power plant in their respective subtrees.

We will also preprocess this Kruskal tree so that we can answer LCA-queries in $O(\log n)$ or $O(1)$ time.

Processing Queries. We process queries in increasing order of r_i (sweep-line). We call a (leaf) node in the Kruskal tree *active* if it has a smaller index than the current r_i we are processing.

Consider an internal node u in the Kruskal tree, let k_u be the maximum index of any active leaf in the subtree rooted at u (this is at most r_i). Also, say a and b are the direct children of u . Then the power line corresponding to u will **not** be built if and only if both $k_a \geq l_i$ and $k_b \geq l_i$. Indeed, this happens only if both the subtree rooted at a and the one rooted at b have some windturbine in the range $[l_i, r_i]$.

We now note that:

$$k_u = \max(k_{u,0}, k_{u,1}).$$

Thus, we need to compute the sum of c of power lines satisfying the above condition efficiently. The critical observation is that this sum is equal to the following:

- the sum of $w_{u.\text{par}} - w_u$ over all u such that $k_u \geq l_i$

Here, u can be an internal node or a leaf node. $u.\text{par}$ denotes the parent of node u (if node u does not have a parent, define $w_{u.\text{par}} = 0$). If u is an internal node, w_u is the cost of the power line corresponding to u . If u is a leaf node, define $w_u = 0$.

For this, we will maintain a Fenwick Tree (or a Segment Tree) indexed by k_u to efficiently compute this sum. In particular, we will maintain, for each ℓ , the sum of $w_{u.\text{par}} - w_u$ over all u with $k_u \geq \ell$.

We can maintain the values of k_u efficiently while performing the sweep-line. When activating a leaf x , the k -values for all nodes from that leaf to the root will be set equal to the index of the newly activated leaf. Along this leaf-to-root path, there were several “chains” that share the same k -value.

Define a *chain* as a path as a maximal path from some internal node u to the leaf k_u . We say the topmost vertex on this chain is the chain's root.

When activating a leaf node x , (currently the largest activated node), this will create a new chain from the root of the tree all the way to x , and override any previous chains on this path, making them shorter.

We can observe that throughout the sweep-line on r , the total number of times chains will change roots is $O(N \log N)$. This is since, at node u with two children subtrees of size s_1 and s_2 , the chain passing through u can be change direction at most $\min(s_1, s_2)$ times. The worst-case is in a balanced binary tree, where the chain at the root changes direction $N/2$ times; the chains through a node at one level below the root changes direction $N/4$ times and so on.

By the above observation we can afford to process each chain-root-change one by one (total $O(N \log N)$), and every time it happens update the affected indices in the Fenwick tree in $O(\log N)$ per update.

These chains can be maintained by simply storing the k_u values for all chain-roots, and then use LCA-queries to traverse the new root-to-leaf path downwards after an activation. Indeed, say we activate node x , then we can find the last overlapping vertex on the intersection of the root-to- x path and the chain starting at u by querying the LCA of x and k_u . (Alternatively, a Heavy-Light Decomposition or other tree data structures can help traverse these paths). We can then keep going on one of the childs of this LCA and process the next chain, while updating the Fenwick tree and chain-roots.

Time complexity: $O(M \log M)$ to sort the edge, $O(Q \log N)$ for the Fenwick tree queries and $O(N \log^2 N)$ activating the leafs and updating the Fenwick tree.